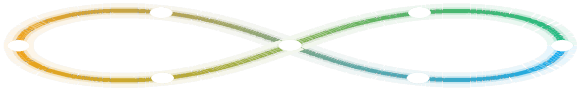

User Manual & Technical Reference

A Tensor-Native, Hardware-Accelerated Laboratory for Multi/Many-Objective Optimization, High-Throughput Benchmarking, and In-Situ MCDM

METISBr — Research Group on Multi-Objective and Many-Objective Evolutionary Optimization

UFOP — Federal University of Ouro Preto, Brazil

Package version: 1.0.2 | Manual edition: 2026.09 | Generated on 2026-09-06

Contact: santostf+metisbr@ufop.edu.br | https://github.com/METISBR/emopylab

Contents

1. Introduction & Core Value Proposition
2. Requirements and Installation
3. Tensor-Native Architecture & Design Principles
4. Repository Directory Structure
5. Core Engine & Native Tensor Classes (core/)
6. Extensible Plugin System
7. Implemented Metaheuristic & Problem Catalog
8. The Scientific Desktop Workstation (PySide6 GUI)
9. Cryptographic Reproducibility (SHA-256 Ledger)
10. Non-Parametric Statistical Suite & In-Situ MCDM
11. Authoring Native Optimization Problems
12. Authoring Native Metaheuristics
13. Creating Tensor Quality Metrics
14. Implementing Genetic & Selection Operators
15. High-Throughput Execution & HPC Workflows (Santos Dumont)
16. Troubleshooting & Operational FAQ
Appendix A. 5-Tier Hardware Backend Dispatch (CUDA / MLX / JAX / CuPy / SIMD)
Appendix B. References & Scientific Citations

1. Introduction & Core Value Proposition

EmoPyLab is an open-source, tensor-native scientific laboratory and visual decision platform engineered specifically for Evolutionary Multi-Objective (EMO) and Many-Objective Optimization (MaOP, $M \geq 2$). The framework provides a completely autonomous, high-performance ecosystem designed to overcome the historical memory fragmentation, interpreter bottlenecks, and object-allocation overheads typical of legacy Python toolchains.

The entire desktop workstation is implemented in `emopylab_app.py` (17,528 lines in PySide6/Qt6), while the underlying high-performance engine is isolated in `core/`. The graphical workstation integrates five dedicated research environments: **Quick Test**, **Experiment**, **Results**, **AI Agent** (featuring local in-process Qwen inference), and **Extensibility**.

1.1. Scientific & Technical Highlights

- **Structure-of-Arrays (SoA) Data Model:** Eliminates per-individual Python heap objects in favor of contiguous memory tensors ($X \in \blacksquare^{N \times D}$, $F \in \blacksquare^{N \times M}$, $G \in \blacksquare^{N \times K}$).
- **5-Tier Hardware Backend:** Transparent, zero-recompilation execution across NVIDIA CUDA, Apple Silicon MLX (Neural Engine), Google JAX (XLA), CuPy, and optimized multicore CPU C-SIMD.
- **High-Throughput Many-Objective Sorting:** Native deductive Efficient Non-Dominated Sorting (ENS, $\blacksquare(M\ N\ \sqrt{N})$) and GPU Boolean Matrix Dominance for massive population regimes ($N \geq 4{,}000$).
- **Quasi-Monte Carlo Hypervolume (Fast-MC HV):** Low-discrepancy Sobol sequence estimator with bounding-box pruning enabling sub-10 ms evaluation in up to 15-objective spaces.
- **Local AI Assistant:** Zero-cloud code generation and mathematical formulation powered by the local quantized `qwen2.5-0.5b-instruct-q4_k_m.gguf` model.
- **In-Situ MCDM & Back-Mapping:** Built-in a posteriori compromise selection (TOPSIS, PROMETHEE II, Compromise Programming) with deterministic back-mapping to decision space X.
- **Rigorous Statistical Suite:** Non-parametric inferential hypothesis testing (Wilcoxon, Friedman with Kendall's W, Vargha-Delaney A_{12} effect size, and Holm-Bonferroni FWER control).
- **Cryptographic Provenance:** Canonical JSON manifests with SHA-256 digests securing deterministic reproducibility across supercomputers and local workstations.

2. Requirements and Installation

EmoPyLab is a pure-Python, standalone tensor-native ecosystem compatible with **Python 3.10, 3.11, and 3.12+**. It is packaged under modern PEP 621 conventions (`pyproject.toml`) and runs across Linux, macOS (Apple Silicon and Intel), and Windows.

2.1. Core Dependencies

Package	Version Constraint	Role in EmoPyLab
numpy	>=2.3, <2.5	Core contiguous numerical tensor arrays
numba	>=0.66, <0.67	JIT-compiled CPU kernels for fast NDS and dominance
scipy	>=1.16, <2.0	Statistical test suite (Wilcoxon, Friedman, A12)
matplotlib	>=3.10, <4.0	Scientific Pareto fronts and convergence plotting
psutil	>=7.2, <8.0	Hardware telemetry and memory bandwidth monitoring

2.2. Optional Acceleration & Extension Groups

Group	Packages	Target Acceleration
gui	PySide6, qt-material, qtawesome	Desktop workstation graphical interface
torch	torch	NVIDIA CUDA / Apple MPS acceleration (Tier 1)
cupy	cupy-cuda12x	NVIDIA CUDA drop-in array operations (Tier 2)
jax	jax, jaxlib	GPU/TPU XLA compilation and 3D vmap tensor batching (Tier 3)
mlx	mlx, mlx-lm	Apple Silicon M-series GPU & Neural Engine (Tier 4)
llm	llama-cpp-python	In-process GGUF local model inference (Qwen 2.5)
dev	pytest, scikit-learn	Automated contract verification and surrogate models

2.3. Clean Installation Commands

```
# 1. Clone clean repository
git clone https://github.com/METISBR/emopylab.git
cd emopylab

# 2. Install base core dependencies
pip install .

# 3. Install acceleration suite
pip install .[gui,llm,jax]        # On Linux / Windows (JAX XLA)
pip install .[gui,cupy]           # On Linux / Windows (NVIDIA CUDA 12.x)
pip install .[gui,mlx,llm]        # On Apple Silicon macOS
```

3. Tensor-Native Architecture & Design Principles

Traditional evolutionary computation toolchains in Python rely on an *Array-of-Structures* (AoS) model, where each individual is represented as a separate Python heap object containing decision vectors, objectives, and metadata. In large-scale campaigns (such as millions of evaluations across hundreds of runs), this model triggers intense memory fragmentation, cache thrashing, and heavy serialization overhead between subprocesses.

EmoPyLab solves this architectural limitation through a pure **Structure-of-Arrays (SoA)** columnar data model. Populations and problems are expressed as contiguous 2D and 3D tensors:

```
# Tensor Population Memory Representation
X in R^(N x D)   # Continuous / discrete decision variable matrix
F in R^(N x M)   # Objective function evaluation matrix
G in R^(N x K)   # Inequality constraint violation matrix
CV in R^(N)      # Total aggregated constraint violation vector
```

3.1. Unified 5-Tier Backend Dispatch Engine

The computational kernel (`core/tensor/backend.py`) auto-detects available accelerators with transparent fallback at runtime without requiring code modification:

- **Tier 1 (NVIDIA PyTorch CUDA / MPS):** Highest throughput for large deep surrogate models and neural architectures.
- **Tier 2 (NVIDIA CuPy):** Direct CUDA array manipulation providing drop-in GPU acceleration for structured array ops.
- **Tier 3 (Google JAX XLA):** Fully vectorizable functional kernels with `jax.jit` and `jax.vmap` across parallel seeds.
- **Tier 4 (Apple MLX):** Native Apple Silicon execution operating directly on unified memory, avoiding PCIe bus transfers.
- **Tier 5 (Vectorized CPU C-SIMD):** OpenBLAS/MKL fallback with AVX-512 vectorization and thread oversubscription protection.

3.2. Subprocess Worker & Memory-Safe Ring Buffer

Execution isolation is maintained by `emopylab_process_worker.py`. Optimization runs execute in dedicated worker processes communicating through memory-mapped shared buffers, preventing Python Global Interpreter Lock (GIL) contention and bounding GUI RAM consumption strictly below 150 MB.

4. Repository Directory Structure

```
emopylab/
|-- emopylab_app.py              # Main PySide6 Scientific Desktop GUI (17,528 lines)
|-- EmoPyLab.py                  # Entrypoint launcher script
|-- emopylab_metadata.py         # Algorithm, problem, and metric catalog descriptors
|-- emopylab_process_worker.py   # Isolated subprocess execution worker
|-- optimize.py                  # Standalone top-level minimization entrypoint
|-- version.py / __init__.py     # Version metadata (v1.0.2)
|-- pyproject.toml               # Modern PEP 621 package and build specification
|-- requirements.txt             # Pip dependency manifest
|-- CITATION.bib                 # Formal BibTeX citation entry
|-- models/                      # Local AI model directory
|   `-- qwen2.5-0.5b-instruct-q4_k_m.gguf  # 491 MB GGUF model tracked via Git LFS
|-- core/                        # Standalone scientific core (Pure Tensors)
|   |-- algorithm.py             #   Native population-based Algorithm base class
|   |-- problem.py / variable.py #   Optimization Problem contract & variable definitions
|   |-- population.py            #   Contiguous Population data structure
|   |-- tensor/                  #   5-tier tensor backends (CUDA, MLX, JAX, CuPy, SIMD)
|   |-- nds/                     #   Deductive ENS & GPU Boolean matrix sorting
|   |-- mcdm/decision.py         #   TOPSIS, PROMETHEE II & Compromise Programming
|   |-- analysis/stat_tests_advanced.py # Wilcoxon, Friedman, A12, DeltaP, IGD+
|   |-- execution/reproducibility.py    # Canonical SHA-256 manifests & seed plans
|   `-- llm/                     #   Local Qwen client and AST-safe formulation
|-- algorithms/                  # 298 metaheuristic solvers (NSGA-II, NSGA-III, MOEA/D...)
|-- problems/                    # 54 benchmark problems (Single, Multi, Many-Objective)
|-- operators/                   # 75 genetic operators (crossover, mutation, repair...)
|-- metrics/                     # 7 tensor quality indicators (Fast-MC HV, IGD+, GD+)
|-- util/                        # Utility shims, dominator logic, reference directions
`-- tests/                       # Complete regression and benchmark test suite
```

5. Core Engine & Native Tensor Classes (core/)

EmoPyLab operates completely independently of external optimization libraries. Its base classes are located in `core/` and provide native vectorized interfaces designed for high throughput.

5.1. core/problem.py — Native Problem Base Class

Every optimization problem derives from `core.problem.Problem`. Evaluation operates directly on 2D NumPy/JAX arrays:

```
from core.problem import Problem
import numpy as np

class MyProblem(Problem):
    def __init__(self, n_var=30, n_obj=2, **kwargs):
        super().__init__(
            n_var=n_var, n_obj=n_obj, n_ieq_constr=0,
            xl=np.zeros(n_var), xu=np.ones(n_var), **kwargs
        )

    def _evaluate(self, X: np.ndarray, out: dict, *args, **kwargs):
        f1 = X[:, 0]
        g = 1.0 + 9.0 * np.sum(X[:, 1:], axis=1) / (self.n_var - 1)
        f2 = g * (1.0 - np.sqrt(f1 / g))
        out['F'] = np.column_stack([f1, f2])
```

5.2. core/algorithm.py — Native Algorithm Base Class

The base `Algorithm` class provides automatic hardware accelerator binding, deterministic seeding, callback hooks, and survival selection:

```
from core.algorithm import Algorithm

class MyAlgorithm(Algorithm):
    def __init__(self, pop_size: int = 100, use_gpu: bool = False, **kwargs):
        super().__init__(use_gpu=use_gpu, **kwargs)
        self.pop_size = pop_size

    def _step(self):
        # Vectorized mating, evaluation, and survival loop
        offspring = self.mating.do(self.problem, self.pop, self.pop_size)
        self.evaluator.eval(self.problem, offspring)
        self.pop = self.survival.do(self.problem, self.pop, offspring, self.pop_size)
```

6. Extensible Plugin System

EmoPyLab features a zero-registration plugin discovery architecture. Any new algorithm, benchmark problem, genetic operator, or metric added to its corresponding directory is dynamically discovered, inspected, and validated at startup.

6.1. Discovery Pipeline

- **algorithms/**: Auto-scans subfolders containing `ALGORITHM_FLAGS` and typed class constructors.
- **problems/**: Auto-categorizes into `single/`, `multi/`, and `many/` suites.
- **operators/**: Auto-discovers genetic components by category: crossover, mutation, sampling, selection, repair.
- **metrics/**: Discovers indicator functions exposing `create_metric(context)`.

6.2. Registry Dataclasses

Dataclass	Key Fields	Operational Purpose
AlgorithmSpec	id, name, source, module, factory, flags	Solver record with scope tags (single, multi, many)
ProblemSpec	id, name, source, module, factory, n_var, n_obj	Problem record with default problem dimensions
MetricSpec	id, name, source, module, factory	Quality indicator specification and reference requirements
OperatorSpec	id, name, source, category, module, class_name	Genetic operator descriptor grouped by category

7. Implemented Metaheuristic & Problem Catalog

Snapshot verified against the repository on 2026-09-06. All components run on the native EmoPyLab execution engine.

7.1. Quantitative Overview

Domain	Verified Count	Architecture & Taxonomy
Algorithms	298 Solvers	Dominance, Decomposition, Indicator, Reference-Vector, SAEA, LLM-based
Problems	54 Formulations	Single (7), Multi (33 suites), Many-objective (9 suites: MaF, DTLZ, WFG)
Operators	75 Modules	Crossover (24), Mutation (12), Repair (12), Sampling (4), Selection (4)
Metrics	7 Modules	Fast-MC HV (Sobol), R2 Indicator, IGD+, GD+, Spacing, Averaged Hausdorff (DeltaP)

7.2. Representative Algorithm Families

- **Dominance-Based:** NSGA-II, SPEA2, BiGE, PESA-II, Micro-GA, Knock-Out EA, E-NSGA-II.
- **Decomposition-Based:** MOEA/D, MOEA/D-DE, MOEA/D-DRA, C-MOEA/D, RPEA, EAG-MOEA/D.
- **Reference-Vector / Angle:** NSGA-III, RVEA, VAEA, MaOEA-CSS, LARC-NSGA3, GCS-MaOEA.
- **Indicator-Based:** IBEA, SMS-EMOA, HypE, MOMBI-II, SRA, AR-MOEA.
- **Surrogate-Assisted (SAEA):** ParEGO, K-RVEA, CSEA, AS-SAEA, SAEA-TL2M, MOEA/D-EGO.
- **LLM Meta-Controlled:** MaACO (Adaptive Ant Colony), LARC-NSGA3 (Reinforced niching).

8. The Scientific Desktop Workstation (PySide6 GUI)

The graphical interface is built with high-contrast typography, dark/light scientific themes, and a decoupled event loop guaranteeing responsiveness during massive simulations.

8.1. The Five Operational Workspaces

- **Quick Test:** Rapid prototyping suite for single-run diagnostics, offering real-time 2D/3D Pareto front projection, camera rotation, convergence curves, and hardware telemetry.
- **Experiment:** High-throughput batch benchmarking orchestration. Allows matrix cross-testing across dozens of algorithms, test problems, seed replications, and evaluation budgets.
- **Results:** Visual analytics suite offering aggregated performance tables, non-parametric statistical tests, convergence trajectories, and high-resolution CSV/LaTeX exporters.
- **AI Agent:** Local mathematical formulation assistant powered by in-process Qwen 2.5 GGUF. Converts plain English into validated Python/JAX problem plugins with strict AST security auditing.
- **Extensibility:** Interactive explorer for manual plugin creation, template scaffolding, and catalog taxonomy inspection.

8.2. Local AI Assistant & AST Security Architecture

The AI Agent operates 100% locally and offline using `qwen2.5-0.5b-instruct-q4_k_m.gguf`. Before any generated Python code is saved or loaded, it must pass a strict **Abstract Syntax Tree (AST)** security gate (`core/llm/formulation.py`):

- Prohibits dangerous calls (`eval, exec, open, __import__, os.system, subprocess`).
- Restricts imports to authorized scientific libraries (`numpy, jax, typing, math, core`).
- Verifies numerical correctness and CPU/JAX output equivalence on small randomized input batches.

9. Cryptographic Reproducibility (SHA-256 Ledger)

To ensure compliance with FAIR scientific principles, EmoPyLab integrates an automated cryptographic provenance engine (`core/execution/reproducibility.py`). Every experimental campaign generates an immutable canonical JSON sidecar manifest containing:

- **Deterministic Seed Schedules:** Canonical arithmetic seed progression ($S_k = 1000 + 17k$) guaranteeing reproducible pseudo-random streams.
- **Hardware & Backend Fingerprint:** CPU model, active GPU/NPU accelerator tier, and thread counts.
- **Software Environment:** Python version, library commit hashes, and compiler flags.
- **Cryptographic SHA-256 Hash:** Digest computed over canonicalized parameters and final Pareto front coordinates.

10. Non-Parametric Statistical Suite & In-Situ MCDM

10.1. Advanced Statistical Testing

EmoPyLab integrates automated inferential statistics (`core/analysis/stat_tests_advanced.py`):

- **Wilcoxon Signed-Rank Test:** Non-parametric pairwise comparison with significance threshold $\alpha = 0.05$.
- **Friedman Global Ranking & Kendall's W:** Multiple-solver rank ordering with concordance analysis.
- **Vargha-Delaney A_{12} Effect Size:** Non-parametric stochastic superiority measure (negligible, small, medium, large).
- **Holm-Bonferroni Post-Hoc Correction:** Step-down sequential adjustment controlling family-wise error rate (FWER).

10.2. In-Situ Multi-Criteria Decision Making (MCDM)

Pareto optimization yields hundreds of non-dominated solutions. EmoPyLab bridges search and practical deployment by embedding native decision support (`core/mcdm/decision.py`):

Method	Mathematical Foundation	Decision-Making Context
TOPSIS	Euclidean distance to ideal best (min) and worst (max)	Balanced compromise with user-defined attribute weights
PROMETHEE II	Net outranking flows ($\Phi = \Phi^+ - \Phi^-$)	Robust pairwise outranking among non-dominated points
Compromise Prog.	Weighted Chebyshev / L_p metric norm	Target-oriented selection under strict preference vectors

All MCDM methods feature **Decision-Space Back-Mapping**, immediately returning the physical decision vector X^* corresponding to the chosen compromise objective trade-off F^*.

11. Authoring Custom Problems, Algorithms & Metrics

11.1. Authoring a Problem (problems/multi/ or problems/many/)

Implement a subclass of `core.problem.Problem`:

```
from core.problem import Problem
import numpy as np

class CustomZDT(Problem):
    def __init__(self, n_var=30, **kwargs):
        super().__init__(n_var=n_var, n_obj=2, xl=0.0, xu=1.0, **kwargs)

    def _evaluate(self, X, out, *args, **kwargs):
        f1 = X[:, 0]
        g = 1.0 + 9.0 * np.sum(X[:, 1:], axis=1) / (self.n_var - 1)
        f2 = g * (1.0 - np.power(f1 / g, 2))
        out['F'] = np.column_stack([f1, f2])
```

11.2. Authoring an Algorithm Plugin (algorithms/)

EmoPyLab discovers algorithms dynamically with zero manual registration. To create a new metaheuristic solver compatible with the desktop GUI, batch runner, and 5-tier hardware acceleration, create a module inside `algorithms/my_algorithm.py`:

```
# algorithms/my_algorithm.py
import numpy as np
from core.algorithm import Algorithm
from core.population import Population
from core.nds.ens import efficient_non_dominated_sort

# Scope tags for GUI auto-categorization: 'single', 'multi', or 'many'
ALGORITHM_FLAGS = {'multi', 'many', 'gpu_compatible'}

class CustomGeneticAlgorithm(Algorithm):
    def __init__(self, pop_size: int = 100, mutation_rate: float = 0.1, **kwargs):
        super().__init__(**kwargs)
        self.pop_size = pop_size
        self.mutation_rate = mutation_rate

    def _initialize_advance(self, infills=None):
        # Initialize population using uniform random continuous variables
        X = np.random.uniform(self.problem.xl, self.problem.xu, size=(self.pop_size, self.problem.n_var))
        self.pop = Population.new('X', X)
        self.evaluator.eval(self.problem, self.pop)

    def _advance(self, infills=None):
        # 1. Selection and Variation
        parents_idx = np.random.randint(0, len(self.pop), size=self.pop_size)
        X_offspring = self.pop.get('X')[parents_idx] + np.random.normal(0, 0.05, size=(self.pop_size, self.proble
        X_offspring = np.clip(X_offspring, self.problem.xl, self.problem.xu)

        # 2. Evaluate offspring population
        offspring = Population.new('X', X_offspring)
        self.evaluator.eval(self.problem, offspring)

        # 3. Environmental Survival Selection via ENS Non-Dominated Sorting
        combined = Population.merge(self.pop, offspring)
        F = combined.get('F')
        fronts = efficient_non_dominated_sort(F)
        survivors_idx = [idx for front in fronts for idx in front][:self.pop_size]
        self.pop = combined[survivors_idx]
```

11.3. Authoring a Quality Metric (metrics/)

To implement a native quality indicator, derive from `metrics.indicators.Indicator` or expose a function `r2_indicator(F, PF, W)`:

```
# metrics/custom_metric.py
import numpy as np

def create_metric(context: dict):
    ref_front = context.get('reference_front')
    def _evaluator(front: np.ndarray) -> float:
        # Compute distance metric to ref_front
        return float(np.mean(np.min(np.linalg.norm(front[:, None] - ref_front, axis=2), axis=1)))
    return _evaluator
```

15. High-Throughput Execution & HPC Workflows

EmoPyLab is engineered for seamless transition from local workstations to high-performance supercomputers. The framework was empirically validated across **5,970 independent runs** on the Santos Dumont Supercomputer (Bull Sequana X1000 GPU partition, LNCC/MCTI, Brazil).

15.1. Headless CLI Optimization

```
# Run single optimization headlessly
emopylab optimize --algorithm NSGA3 --problem DTLZ2 --n-obj 5 --seed 42

# Launch automated matrix benchmark from JSON specification
emopylab benchmark --config benchmark_matrix.json --workers 36 --output results/
```

15.2. Slurm Supercomputing Deployment

Cluster scripts (`sbatch_sdumont_benchmark.sh`) prevent thread oversubscription by strictly setting BLAS thread pools (`OPENBLAS_NUM_THREADS=1`, `MKL_NUM_THREADS=1`) prior to dispatching across massive GPU/CPU worker pools.

Appendix A. Hardware Acceleration Backend Architecture

Backend Tier	Module Dispatcher	Platform Support	Optimization Focus
Tier 1: CUDA / MPS	core.tensor.backend (torch)	NVIDIA GPUs / Apple Silicon MPS	Deep surrogates & high VRAM batches
Tier 2: CuPy	core.tensor.backend (cupy)	NVIDIA CUDA Architecture	Drop-in structured array acceleration
Tier 3: Google JAX	core.tensor.backend (jax)	Linux, macOS, Windows (XLA)	Vectorized multi-seed parallel vmap
Tier 4: Apple MLX	core.tensor.backend (mlx)	macOS Apple Silicon (ARM64)	Zero-copy unified memory & Neural Engine
Tier 5: CPU C-SIMD	core.tensor.backend (numpy)	Universal x86_64 / ARM64	Optimized OpenBLAS/MKL AVX-512 fallback

Appendix B. References & Citation

If you utilize EmoPyLab in academic or industrial research, please cite:

```
@article{santos2026emopylab,
  author  = {Santos, Thiago and Xavier, Sebasti{\~a}o},
  title   = {{EmoPyLab}: A Tensor-Native, Hardware-Accelerated Laboratory for High-Throughput Benchmarking and De
  journal = {Available at SSRN},
  year    = {2026},
  doi     = {10.2139/ssrn.7412768},
  url     = {https://ssrn.com/abstract=7412768},
  note    = {Available at SSRN: \url{https://ssrn.com/abstract=7412768} or \url{http://dx.doi.org/10.2139/ssrn.74
}

@software{Santos_EmoPyLab_2026,
  author  = {Santos, Thiago and Xavier, Sebasti{\~a}o},
  title   = {{EmoPyLab: A High-Throughput Benchmarking Ecosystem and Open-Source Decision Platform for Evolutiona
  month   = {9},
  year    = {2026},
  version = {1.0.2},
  url     = {https://github.com/METISBR/emopylab},
  license = {MIT}
}
```